Informe Post-Incidente — Unidad 6: Food Store (PostgreSQL 16+)

Integrantes: Valentini, Tiziano — García, Juan Martín | **Motor:** PostgreSQL 16+ (verificado en PostgreSQL 18.3, Windows / PowerShell)
Base principal (intacta): `foodstore` (179 MB) | **Base descartable de simulacro:** `foodstore_restore_test` | **Evidencias:** `evidencias/` y `logs/`

1. Respaldo Utilizado y Métricas del Simulacro (RPO vs. RTO)

- **Archivo de respaldo utilizado:** `backups/foodstore_20261109_221751.dump` (formato Custom `-Fc`, generado por `backup_foodstore.ps1` en **1,950 s**, ExitCode = 0).
- **Fecha y hora del respaldo:** `2026-11-09 22:17:53.221 -03:00` | **Tamaño:** 8.433.690 bytes (8,04 MB) | **SHA-256:** `349A7FC7...49AA0B`.
- **Inicio del simulacro (`restore_test.ps1`):** `2026-11-09 22:17:54.802 -03:00` (sobre `foodstore_restore_test`, sin tocar `foodstore`).

Métrica	Definición conceptual	Previsto (Meta)	Observado	Desglose e interpretación técnica
RPO *(Recovery Point Objective)*	Pérdida máxima de datos tolerable medida **hacia atrás** en el tiempo (antigüedad del respaldo al ocurrir el incidente).	**<= 24 horas** (respaldo diario 02:00 AM)	**1,58 segundos** (`00h:00m:01s` / 0,03 min)	En el ensayo se restauró la copia recién creada. En producción con `pg_dump` diario, el RPO máximo es de 24 h.
RTO *(Recovery Time Objective)*	Tiempo máximo tolerable de interrupción medido **hacia adelante** desde el inicio de la restauración hasta validar.	**<= 300 segundos** (5 minutos)	**5,183 segundos** (5182 ms)	`createdb`: 0,250 s (250 ms) `pg_restore`: 4,214 s (4214 ms) Validación SQL: 0,718 s (718 ms).

2. Resultado de las Validaciones de Tablas, Restricciones Lógicas y Mantenimiento

Tabla (Validación 1)	Filas foodstore	Filas restore_test	Estado	Controles lógicos (Validaciones 2-3) y Mantenimiento (Parte 3)	Resultado
`categoria`	12	12	**OK (100%)**	`productos_invalidos` (`precio_actual < 0 OR stock < 0`)	**0 filas**
`cliente`	20.003	20.003	**OK (100%)**	`detalles_invalidos` (`cantidad <= 0 OR precio_unitario < 0`)	**0 filas**
`producto`	50.005	50.005	**OK (100%)**	`VACUUM (VERBOSE, ANALYZE)` en `detalle_pedido` (23.974 → 0 muertas)	**0,00%**
`pedido`	200.003	200.003	**OK (100%)**	`VACUUM (VERBOSE, ANALYZE)` en `pedido` (7.719 → 0 muertas)	**0,00%**
`detalle_pedido`	400.011	400.011	**OK (100%)**	`VACUUM (VERBOSE, ANALYZE)` en `producto` (3.450 → 0 muertas)	**0,00%**

3. Problemas Reales Encontrados durante el Procedimiento y su Resolución

1. **Discrepancia de columna (`precio` vs. `precio_actual`):** La consulta de referencia del enunciado filtraba `WHERE precio < 0 OR stock < 0` en `producto`, pero en nuestro esquema real (`TP1_FoodStore/schema.sql`) la columna es `precio_actual`. Se detectó en la revisión humana y se ajustó a `WHERE precio_actual < 0 OR stock < 0`.
2. **Omisión de barrido de índices con `INDEX_CLEANUP AUTO` en PostgreSQL 16+:** Al correr `VACUUM (VERBOSE, ANALYZE) detalle_pedido`, PostgreSQL omitió la limpieza de índices porque las páginas con punteros muertos representaban el 1,63% (< 2% de umbral de bypass). Se resolvió usando `VACUUM (VERBOSE, ANALYZE, INDEX_CLEANUP ON)`, reduciendo `n_dead_tup` a 0 sin usar `VACUUM FULL` (evitando bloqueos `ACCESS EXCLUSIVE`).
3. **Codificación en Windows PowerShell 5.1:** La lectura sin BOM en CP1252 interpretaba bytes UTF-8 acentuados como comillas de cierre. Se normalizaron los scripts `.ps1` y se configuró `PGCLIENTENCODING=UTF8`.

4. Mejora Concreta para Reducir Riesgo y Tiempo en la Siguiente Ejecución

- **Reducción de RTO:** Incorporar paralelismo `pg_restore -j 4` sobre el respaldo en formato `-Fc`, paralelizando la carga de tablas y reconstrucción de índices B-Tree en `pedido` y `detalle_pedido`.
- **Reducción de RPO y bloat:** Combinar el `pg_dump` diario con archivado continuo de WAL (*PITR*) para reducir el RPO de 24 h a minutos, y ajustar `ALTER TABLE detalle_pedido SET (autovacuum_vacuum_scale_factor = 0.05);` (5% en vez de 20%).

5. Uso de OpenCode y Verificación Humana

- **Asistencia con OpenCode:** A partir de `SPEC_U6.md`, se generó con OpenCode la estructura inicial de `backup_foodstore.ps1`, `restore_test.ps1` (con `System.Diagnostics.Stopwatch`) y la consulta diagnóstica de `maintenance.sql`.
- **Verificación humana previa a la aceptación:** Se revisó cada script confirmando: **(a)** cero contraseñas expuestas ni variables `$env:PGPASSWORD` (usando `%APPDATA%\postgresql\pgpass.conf`); **(b)** uso de `producto.precio_actual`; **(c)** exclusión de `VACUUM FULL` e inclusión de `INDEX_CLEANUP ON`; y **(d)** ejecución real verificando que `foodstore` permaneció intacta.